

bigTpuProfile

bigTpuProfile 是一个板卡性能可视化的工具（目前仅支持bm1690）。

目录

- [bigTpuProfilee](#)
 - [目录](#)
 - [快速开始](#)
 - [profile数据导出](#)
 - [算子](#)
 - [tpu-train/tgi](#)
 - [tpudnn](#)
 - [bmodel](#)
 - [可视化](#)
 - [数据分析](#)
 - [文件夹结构](#)
 - [doc](#)
 - [web](#)
 - [许可证](#)

快速开始

bigTpuProfile的使用主要分为两个步骤：

1. profile数据导出
2. 通过tpu-mlir进行可视化

profile数据导出

算子

1. bigTpuProfile有三种模式：
 - 1) 模式0 pmu only：不关心算子中具体cmd的类型，只关心时间维度，该模式对性能影响最小
 - 2) 模式1 精简cmd：关心算子中各cmd的类型，该模式对性能影响较小，dma带宽统计不准确（额外开销 ~4%）
 - 3) 模式2 详细cmd：关心算子中各cmd的详细信息，通常用于调试、各dma带宽统计（额外开销 7 ~ 10%）
2. max_record_num指的是profile的最大记录条数，需注意设置的值要大于记录值。
3. profile输出文件命名规则：

cdm_profile_data_dev{DeviceID}-{CallNum}

profile 可在多个设备上独立运行文件名中标记为DeviceID, 也可在该设备上多次被调用(CallNum)

tpu-train/tgi

```
torch.ops.my_ops.enable_profile(max_record_num, mode) # 设置记录起始点(记录cmd信息,
mode: 0 pmu only, 1 精简cmd, 2 详细cmd)
```

```
torch.ops.my_ops.disable_profile() # 设置记录结束点
```

```
# tpu-train example
# part 0
torch.ops.my_ops.enable_profile(max_record_num, 0) # enable profile without cmd
info (pure pmu
_ = a tpu * b tpu
torch.ops.my_ops.disable_profile() # disable profile and dump data
(cdm_profile_data_dev0-0)
# part 1
torch.ops.my_ops.enable_profile(max_record_num, 1) # enable profile with
condensed cmd info
_ = a tpu + b tpu
torch.ops.my_ops.disable_profile() #(cdm_profile_data_dev0-1)
# part 2
torch.ops.my_ops.enable_profile(max_record_num, 2) # enable profile with
detailed cmd info
_ = a tpu + b tpu
torch.ops.my_ops.disable_profile() #(cdm_profile_data_dev0-2)
```

```
# tgi (text-generation-inference) example
# test_whole_parallel.py

def test_whole_model(batches=1, model_id="llama", model_path="/data",
quantize=None, mode="chat"):
    ....
    for it in range(2):
        ....
        for i in range(DECODE_TOKEN_LEN):
            os.environ["TOKEN_IDX"] = str(i)
            generate_start = time.time_ns()
            generations, next_batch, (forward_ns, decode_ns) =
model.generate_token(
                next_batch
            )
            generate_end = time.time_ns()
            time_list.append(generate_end - generate_start)
            for generation in generations:
                if i == 0:
                    generated_text[generation.request_id] =
generation.tokens.texts[0]
                else:
                    generated_text[generation.request_id] +=
generation.tokens.texts[0]
                if decode_only and enable_profile and it > 0 and i == 0:
# condition
                    torch.ops.my_ops.enable_profile(max_record_num, book_keeping)
# enable profile
                    logger.info(f"Token {i} {[g.tokens.texts[0] for g in generations]}")
                    if next_batch is None:
                        break
```

```

        if enable_profile and it > 0:
            # condition
            torch_tpu.tpu.optimizer_utils.OpTimer_dump()
            torch.ops.my_ops.disable_profile()
            # disable profile
    .....
```

tpudnn

```

// 假定handle类型为: tpudnnHandle_t
auto pimpl = static_cast<TPUDNNImpl *>(handle);

pimpl->enableProfile(max_record_num, mode); // # 设置记录起始点(记录cmd信息, mode:
0 pmu only, 1 精简cmd, 2 详细cmd)
pimpl->disableProfile(); // 设置记录结束点
```

```

// tpudnn example
....
const int group_num = 1;
const int group_size = pimpl->getCoreNum();
pimpl->enableProfile(); // enable profile
status = pimpl->launchKernel("gelu_forward_multi_core", &api, sizeof(api),
group_num, group_size);
pimpl->disableProfile(); // disable profile
pimpl->enableProfile(80); // enable profile
status = pimpl->launchKernel("gelu_forward_multi_core", &api, sizeof(api),
group_num, group_size);
pimpl->disableProfile(); // disable profile
return status;
```

bmodel

与针对算子的使用方式不同，bmodel主要通过环境变量进行控制，仅需关注最大记录条目数是否合适，无需设置mode模式：

- `ENABLE_ALL_PROFILE=1`：启动profile
- `TPUKERNEL_FIRMWARE_PATH=/home/xxx/libfirmware_core.so`：设置firmware.so 若bmodel版本太老需设置
- `PROFILE_MODE`：（可选项，可选值1 2）查看额外的profile信息，如input, output搬运信息
- `PROFILE_RECORD_SIZE`：（可选项，默认值131072）设置 pmu 最大记录的条目数

```

# 用默认的libfirmware_core.so和条目数
ENABLE_ALL_PROFILE=1 tpu-model-rt --bmodel ./xxx.bmodel

# 指定firmware.so, 指定记录条目数, 查看inputs ouputs搬运性能
TPUKERNEL_FIRMWARE_PATH=/home/xxx/libfirmware_core.so ENABLE_ALL_PROFILE=1
PROFILE_MODE=2 PROFILE_RECORD_SIZE=40960 tpu-model-rt --bmodel ./xxx.bmodel
```

可视化

通过bigTpuProfile进行解析与可视化

```
# 使用 bigTpuProfile -h 查看可用参数
bigTpuProfile cdm_profile_data_devX-X/ result_out --arch BM1690 # 可视化结果存储于
result_out
```

数据分析

文件夹结构

```
/result_out/
├─ tiuRegInfo_x
├─ tdmaRegInfo_x
├─ cdmaRegInfo_x
├─ PerfDoc    文档可视化结果
├─ PerfWeb    web可视化结果
```

doc

PerfAl_output.xlsx 中包括总览，及各个core中各engine（tiu gdma sdma cdma）所记录到的有效数据

命名规则为：engineType_coreId

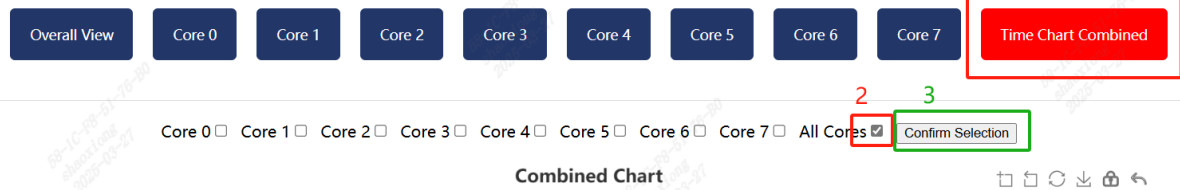
5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33
Engine	Id	Core Id	Global Id	Cmd Id	Id	Layer	Idayer	Namection	Thction	Ndata	sizeart	Cyc	End Cyc	Cyclic	Cyc	Lall	Cyc	andwidth	andwidth	Direction	BurstLen	Data	Type					
7	3	0	0	5	-	DMA_tens	tensor	21504	617314	617903	590	0	36.4475	0	DDR->L2M	0	FP16											
8	3	0	1	6	-	DMA_sys	sys_send	0	617906	617917	12	0	0	0	-	0	None											
9	3	0	2	7	-	DMA_sys	sys_wait	0	617920	618247	328	0	0	0	-	0	None											
10	3	0	3	8	-	DMA_sys	sys_send	0	618250	618261	12	0	0	0	-	0	None											
11	3	0	4	9	-	DMA_sys	sys_wait	0	618264	636823	18560	0	0	0	-	0	None											
12	3	0	5	10	-	DMA_sys	sys_send	0	636826	636837	12	0	0	0	-	0	None											
13	3	0	6	11	-	DMA_sys	sys_wait	0	636840	641632	4793	0	0	0	-	0	None											
14	3	0	7	12	-	DMA_gath	DMA_gath	73728	641678	643801	2124	0	34.7119	0	DDR->L2M	0	FP16											
15	3	0	8	13	-	DMA_gath	DMA_gath	1536	643803	644370	568	0	2.7042	0	DDR->L2M	0	FP16											
16	3	0	9	14	-	DMA_sys	sys_send	0	644373	644384	12	0	0	0	-	0	None											
17	3	0	10	15	-	DMA_gath	DMA_gath	73728	644430	646275	1846	0	39.9393	0	DDR->L2M	0	FP16											
18	3	0	11	16	-	DMA_gath	DMA_gath	1536	646277	646783	507	0	3.0296	0	DDR->L2M	0	FP16											
19	3	0	12	17	-	DMA_sys	sys_send	0	646786	646797	12	0	0	0	-	0	None											
20	3	0	13	18	-	DMA_gath	DMA_gath	73728	646843	648478	1636	0	45.066	0	DDR->L2M	0	FP16											
21	3	0	14	19	-	DMA_gath	DMA_gath	1536	648480	649292	813	0	1.8893	0	DDR->L2M	0	FP16											
22	3	0	15	20	-	DMA_sys	sys_send	0	649295	649306	12	0	0	0	-	0	None											
23	3	0	16	21	-	DMA_gath	DMA_gath	73728	649352	651107	1756	0	41.9863	0	DDR->L2M	0	FP16											
24	3	0	17	22	-	DMA_gath	DMA_gath	1536	651109	651644	536	0	2.8657	0	DDR->L2M	0	FP16											
25	3	0	18	23	-	DMA_sys	sys_send	0	651647	651658	12	0	0	0	-	0	None											
26	3	0	19	24	-	DMA_sys	sys_send	0	651661	651672	12	0	0	0	-	0	None											
27	3	0	20	25	-	DMA_sys	sys_wait	0	651675	658831	7157	0	0	0	-	0	None											
28	3	0	21	26	-	DMA_sys	sys_send	0	658834	658845	12	0	0	0	-	0	None											
29	3	0	22	27	-	DMA_sys	sys_wait	0	658848	659084	237	0	0	0	-	0	None											
30	3	0	23	28	-	DMA_tens	tensor	10752	659130	659627	498	0	21.5904	0	DDR->L2M	0	FP16											
31	3	0	24	29	-	DMA_sys	sys_send	0	659630	659641	12	0	0	0	-	0	None											
32	3	0	25	30	-	DMA_sys	sys_wait	0	659644	659749	106	0	0	0	-	0	None											
33	3	0	26	31	-	DMA_sys	sys_send	0	659752	659763	12	0	0	0	-	0	None											
		Engine Summary	TIU 0	GDMA 0	SDMA 0	TIU 1	GDMA 1	SDMA 1	TIU 2	GDMA 2	SDMA 2	TIU 3	...	+	-	+	-	+	-	+	-	+	-	+	-	+	-	

web

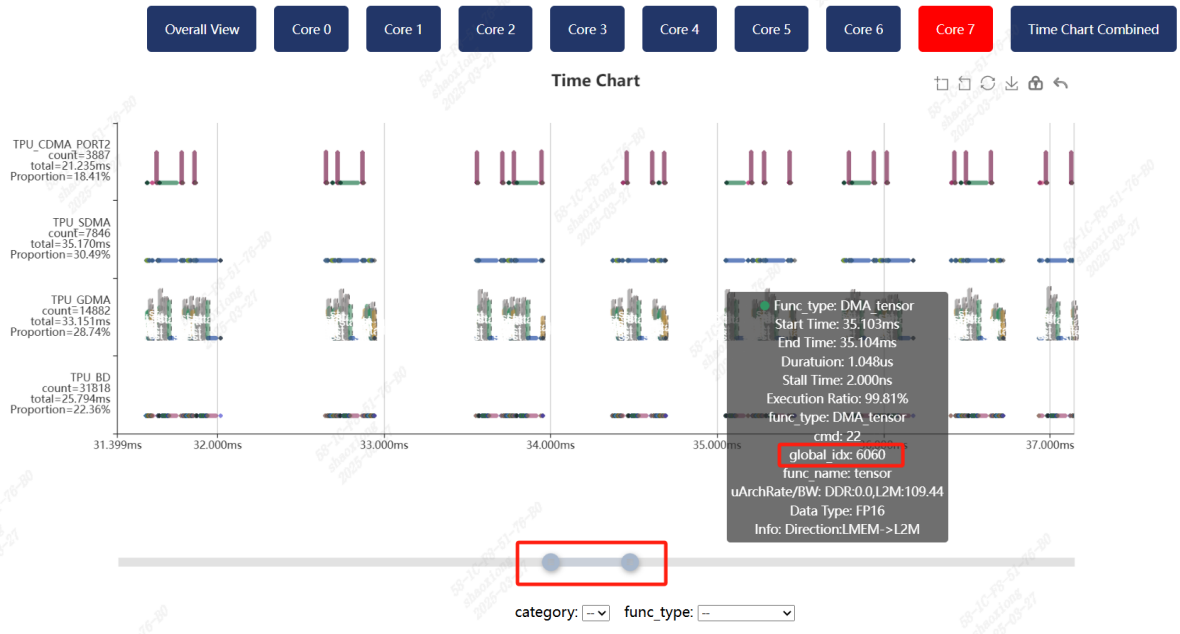
result.html 中包括总览，各个core的可视化，以及core间的比対图

PerfAI: PerfAI_web

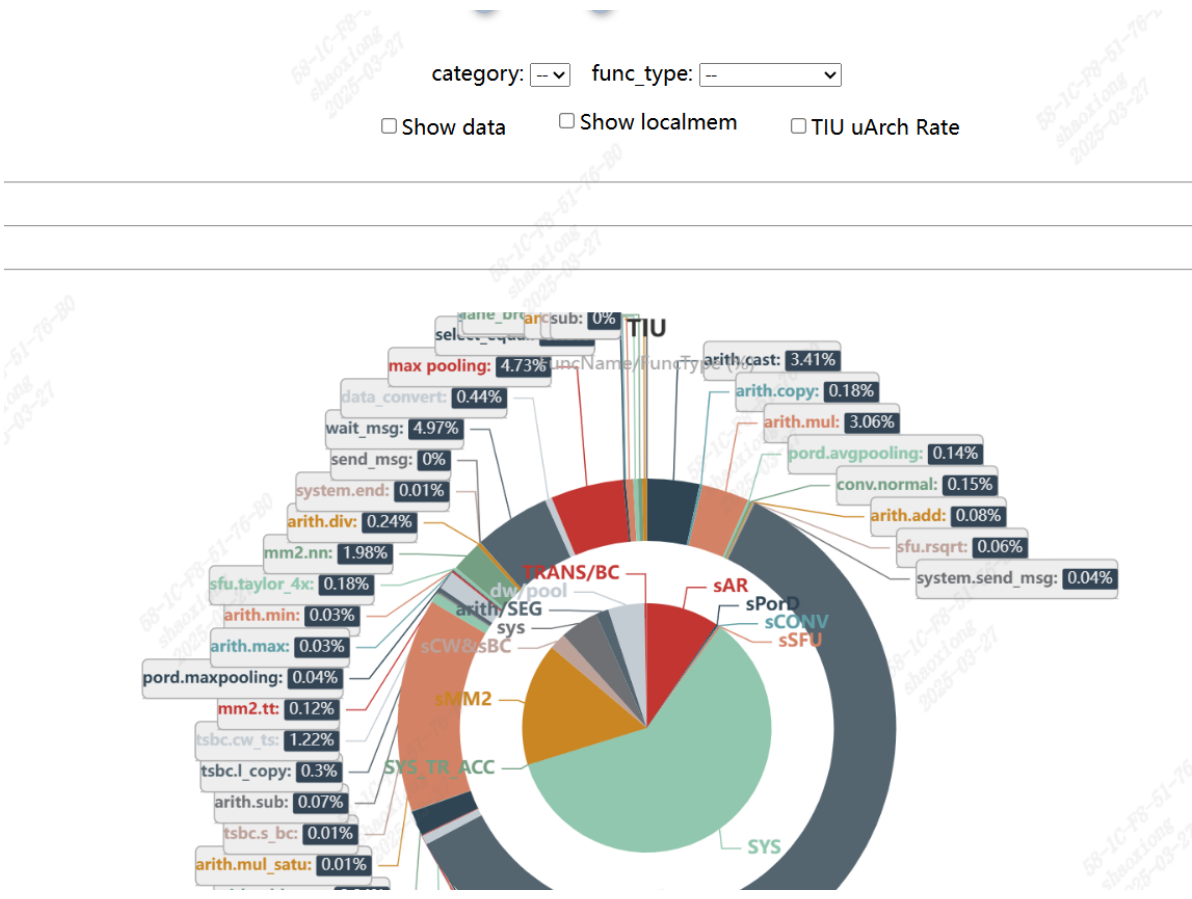
Platform: sg2260



web中可通过滚轮或滑块进行局部缩放，悬停到某条指令后可通过global_idx，在doc对应的engineType_coreId子栏中的C列找到对应指令的详细信息



通过其他选项也有助于性能分析（TIU uArch Rate 功能暂未支持）



许可证

bigTpuProfile 采用 2-Clause BSD 许可证，但第三方组件除外。